

Downtime Optimized Live Migration of Industrial Real-Time Control Services

Michael Gundall*, Julius Stegmann*, Mike Reichardt*, and Hans D. Schotten*[†]

*German Research Center for Artificial Intelligence GmbH (DFKI),

Kaiserslautern, Germany

[†]Department of Electrical and Computer Engineering, Technische Universität Kaiserslautern,

Kaiserslautern, Germany

Email: {michael.gundall, julius_raphael.stegmann, mike.reichardt, hans_dieter.schotten}@dfki.de

Abstract—Live migration of services is a prerequisite for various use cases that must be fulfilled for the realization of Industry 4.0. In addition, many different types of services need to provide mobility and consequently need to be migrated live. These can be offloaded algorithms from mobile devices, such as unmanned vehicles or robots, security services, communication services or classic control tasks. In particular, the latter place very high demands on determinism and latency. Here, it is of utmost importance that the downtime of the service is as low as possible. Since existing live migration approaches try to optimize multiple metrics such as downtime, migration time as well as energy consumption, which are equally relevant in the IT domain, it is not possible to use any of these approaches without adoptions.

Therefore, a novel concept is proposed that builds on top of both existing migration approaches as well as virtualization technologies and aims primarily at minimizing service downtime. Furthermore, the concept is evaluated using a test environment. The results show that a sub-millisecond downtime can be achieved with the proposed concept. Moreover, the total migration time is in the range of several hundred milliseconds for the highest performance setting and two seconds for a non-invasive approach.

Index Terms—Live migration, Redeployment, container, control services, real-time, industrial automation, IT-OT convergence, mobility, flexibility

I. INTRODUCTION

Digitalization enables opportunities and benefits for a broad range of applications and use cases [1], [2]. Furthermore, in manufacturing and process industries the Industry 4.0 vision was introduced [3]. It describes scenarios and goals for highly digitized future factories. These goals are a shortened "time to market", increased resource efficiency, and customization of products [3]. Looking deeper into the individual goals, each of them requires a very high flexibility of the factory modules. In order to increase the flexibility new, predominantly wireless, communication standards such as 5th Generation (5G) [4] and 6th Generation Wireless Communication Systems (6G) [5] as well as powerful data processing tools are investigated.

Moreover, so-called Automated Guided Vehicles (AGVs) will increasingly move through industrial plants. Here, the offloading of algorithms and the interaction between different AGVs or AGV and factory modules require an increased networking between these devices. Very often, also stringent demands on Quality of Service (QoS), such as latency, are required. Thus, the offloaded data has to be processed at servers close to the

devices, so-called edge servers, instead of transferring it through the whole Information Technology (IT) backbone to central data servers. This results in more and more distributed and networked systems.

Hence, distributed systems will play a decisive role in the future, where it can often make sense to migrate the associated processes from one to another processing node. If the productivity should not be reduced, this should be done "live" that means without interruption of the normal operation of the plant. In this way, highly flexible, resilient, and low-latency applications can be realized. How this can ideally work without affecting its underlying functionality is to be investigated, with a focus on industrial control processes.

Accordingly, the following contributions can be found in this paper:

- Analysis and assessment of existing live migration approaches for application in industrial applications
- Introduction of a novel downtime optimized live migration concept especially dedicated to fulfill industrial requirements

Therefore, the paper is structured as follows: Sec. II lists different situations where live migration is required, while Sec. III introduces metrics that are related to live migration in industrial environments. Further, a short description of existing live migration concepts and related work on this topic can be found in Sec. IV and Sec. V, respectively. Then, our improved concept is proposed (Sec. IV) and evaluated (Sec. VII). Finally, a conclusion is given (see Sec. VIII).

II. MIGRATION GOALS

Here it can be useful to transfer a process to another compute node for various reasons. Therefore, Tab. I lists the most important use cases and the corresponding goals that are relevant for industrial applications.

The first use case that is well-known in IT data centres, is the migration of a process for energy savings and cost efficiency. As the load on the different servers could change, it can be beneficial to orchestrate the processes in this way that, for example, one of the servers could be spared. Therefore, both energy and money could be saved.

Next, it is possible that hardware has to be changed or replaced. Typically, this is done during a specific maintenance

Tab. I
OVERVIEW OF DIFFERENT MIGRATIONS USE CASES, CORRESPONDING MIGRATION GOALS AND SOURCES OF RELATED INVESTIGATIONS

Migration use cases	Migration goals	Sources
Efficiency, energy consumption	Reduced communication power, reduced computation power	[6], [7]
Interruption-free hardware replacement	Change of hardware, planned maintenance cycles, hardware upgrade	[6], [8]
Availability, resilience, self-healing	Reaction to decreased QoS, reaction to decreased security level	[9], [10]
Mobility support	Mobility of moving unmanned vehicles, movement of persons with mobile devices	[7], [9], [11]
(Seamless) reconfiguration and program updates	Lot size one, increase of product quality, reaction to decreased security level	[8]–[10], [12]

phase, however, replacement of hardware components could also be required during operational phase. This is especially true, if parts break that do not directly cause a failure of the process but are related to it. Then, the process should be migrated to a different hardware node and the hardware can be replaced.

Further, preventing a failure of the process is a reason why a migration should be performed. If any error occurs, such as QoS degradation or as security measure for compromised systems, a reallocation of one or more processes can also be required.

The same counts for mobile devices, such as AGVs. If, for example, such modules move between different production halls with different IT infrastructures, it is necessary to migrate the associated processes to a different computation node. In this way, optimal QoS, such as minimal latency, can always be achieved. Last, live migration enables not only the redeployment of processes but also reconfiguration and rolling updates. If massive flexible production should be realized, with lot size one as most extreme example, this use case is of very high interest. However, live migration concepts are only part of seamless program updates, since also convergence criteria have to be investigated in detail.

III. METRICS

In this section, most important metrics are to be defined. Here, the focus lies on services that are suitable for industrial environments. Further, effects on the process or application itself as well as on the executing infrastructure must be taken into account. Since [6] already describes most important metrics, they are also the basis for this work.

A. Process Downtime

The process downtime T_D is the most important property for the process itself, especially for real-time (RT) applications. Here, the sum of the inactive time intervals of the process during the migration must be considered, since even many short interruptions can have serious effects on the QoS. Even more important, is the longest continuous interval of inactivity. This determines whether, for example, a real-time criterion can be met. For scheduled processes that are required for isochronous control tasks having a fixed sampling time, the triggering of the

migration immediately after a CPU processing phase must also be taken into account.

B. Migration time

The migration time T_M also plays an essential role. The longer the migration time, the more computation and communication resources are used and blocked for other processes. In addition, this metric is the main indicator for the maximum flexibility that could be reached. A process could not migrate more often than this value. This is important, if a mobile device, such as a drone, is changing factory halls in very short intervals.

C. Transmitted data volume

The transmitted data volume D is specified as the amount of data that has to be transferred during the whole migration phase. This value might be less important for one small data set and empty networks, however, network capacity in industrial environments is limited. In most of the cases, a real industrial network is heavily loaded. The more processes have to be migrated in parallel, the more important this parameter becomes.

D. Resource load

In addition to communication system, the resource load L of the host systems must also be considered. Furthermore, an increase of this value also increases the overall energy consumption. Especially in the area of edge and fog computing, where the computational resources are higher compared to field level, but still limited, the relationship between costs and benefits of virtualisation and live migration of processes plays a decisive role.

IV. EXISTING LIVE MIGRATION APPROACHES

This section presents the current state of the art in terms of live migration, which is mostly applied in the IT environment nowadays. Here, the Checkpoint/Restore (C/R) method is widely used. As the name indicates, the process is frozen and its status on the disk is checkpointed. Next, the data is transferred to a new target system and the process can be resumed. Recently, there is a development towards user-space-based approaches, since advantages of high transparency in combination with small invasive intervention in the central components of the operating

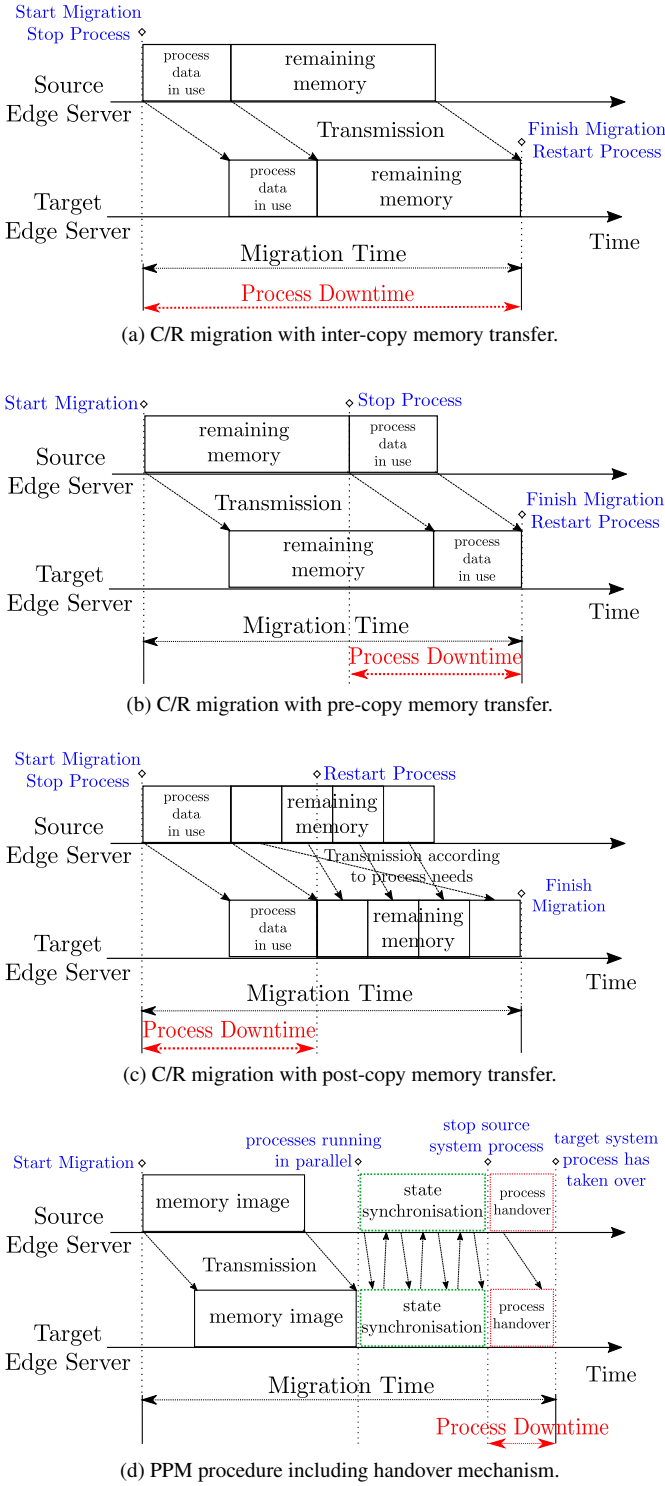


Fig. 1. Overview of the most common live migration approaches that are mostly used in IT environments until now [11].

system can be reached. In order to optimize specific metrics, several main C/R approaches evolved. These are shown in Fig. 1 and discussed in the following part.

A. Inter-copy C/R

The classical (inter-copy) C/R procedure, which is shown in Fig. 1a, consists of the following steps. Firstly, the process is stopped and all data in the memory is completely transferred from one system to another. Then, the process is resumed on the target system. Accordingly, for this procedure, it can be assumed that $T_M = T_D$.

B. Pre-copy C/R

Fig. 1b shows the pre-copy tactic that aims at reduced process downtime. To achieve this goal, as much data as possible is transferred to the target system, before the process is frozen. Here, primarily data that is not expected to be needed in the coming process iterations is selected. Then the process is shut down on the source system, the remaining data is transferred, and the process is restarted on the target system.

C. Post-copy C/R

With the post-copy tactic on the other side, which is depicted in Fig. 1c, the process is directly frozen at the beginning of the migration process, comparable to inter-copy C/R. Afterwards, however, only the parts of the memory that are important for the next process iterations are transferred. The remaining parts of the memory are then transferred after the process has already restarted on the target system [13]. This approach aims at a shorter migration time, but has a higher process downtime compared to pre-copy tactic. Both strategies are part of intensive research [14]–[16]. The post-copy strategies in particular increase the risk of a complete process failure if missing data cannot be transferred in time afterwards. The pre-copy strategy brings few advantages in terms of downtime if large parts of the data change in just a few process steps. Both methods require additional precise prediction of future steps.

D. Hybrid C/R

Hybrid C/R is a combination of pre- and post-copy tactics. Here, most likely required memory pages are transferred to the target system before the process is stopped and transferred. Then the process is already resumed on the target system before the remaining memory pages are transferred. This approach is a good choice if both migration time and process downtime should be minimized, since the process downtime is only slightly higher compared to pre-copy and the migration time is only slightly higher compared to the post-copy tactic, and vice versa. With all the aforementioned approaches, however, the process downtime is still too high for industrial services, where mostly only outages of <1ms are acceptable.

E. Parallel Process Migration (PPM)

Therefore, latest approaches go one step further and use the parallel migration methodology [17], [18] that is also referred to as PPM [11]. In previous approaches, only one instance of the process was active at a time. Thus, Fig. 1d, describes the idea that the process is already running on the target system while both processes are supplied with the same data. When a migration is triggered, only a very small part of the memory has

Tab. II
QUALITATIVE COMPARISON BETWEEN EACH OF THE PROPOSED
MIGRATION APPROACHES AND THE CORRESPONDING METRICS.

Migration Approach	Metrics			
	T_D	T_M	D	L
Inter-copy C/R	-	++	+	+
Pre-copy C/R	+	o	o	o
Post-copy C/R	o	+	o	o
Hybrid C/R	+	+	-	o
Parallel Process Migration	++	o	-	-

to be transferred to the target system. This results in significantly decreased process downtime [17], [18]. Nevertheless, there are several key challenges to ensure a smooth handover, such as time and state synchronization, while also ensuring that all instances of the processes running in parallel are always supplied with the same data at the same time.

F. Analysis

Each of the proposed live migration approaches has its benefits in one category and corresponding drawback in another. Thus, Tab. II maps the aforementioned metrics to each of the approaches.

First, inter-copy C/R is most simple and requires only a small time for the whole migration process, since the process is immediately stopped, migrated, and resumed. As a result of this, inter-copy C/R is the most efficient method. However, the fact that the process downtime is equally high as the migration time, makes this approach not applicable for industrial control processes. Since the pre-copy approach already transfers process states to the migration destination before the process is checkpointed, the process downtime is decreased. This leads to a longer overall migration time, a higher data volume that is transmitted, and a higher load on source and destination system. In contrast, the post-copy method copies the process states in the end of the migration procedure. This approach is especially suitable if the migration time should be minimized, but inter-copy approach cannot be used due to the very high downtime. We believe that there are certain use cases, where this method is a good tradeoff, even if the process downtime is still too high for industrial control processes. Hybrid C/R combines both aforementioned approaches, but the advantages of this approach become only visible if the processes and corresponding sizes exceed certain limits. As there is a trend to use a lot of microservices instead of huge monolithic processes, this approach is not beneficial compared to either pre-copy or post-copy tactic if only small processes should be migrated. Moreover, it is not always trivial to determine which parts of the memory are urgently needed and which are not. As already mentioned, a reduced process downtime is the major goal of PPM approach. Even if all other metrics are worse compared to all other approaches this is most suitable for the live migration of industrial control services. Thus, this approach serves as basis for several investigations [7], [9].

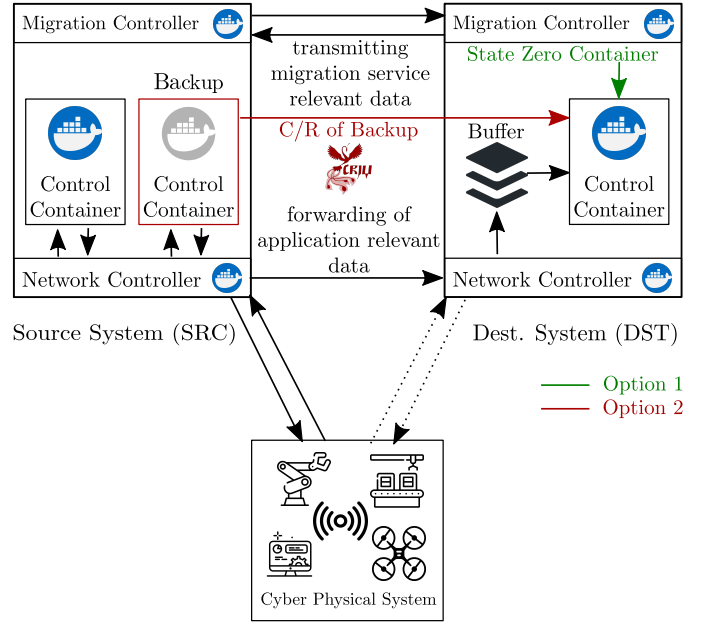


Fig. 2. Novel concept for live migration of industrial control processes with focus of minimized downtime.

V. RELATED WORK

Live migration is already investigated for nearly two decades [19]. The authors in [20], for example, prove that pre-copy tactic leads to a reduced process downtime compared to inter-copy C/R. Moreover, the benefit is even high enough that the process downtime of migrating an Virtual Machine (VM) with pre-copy is lower than migrating a container with inter-copy, even though containers have much lower overhead compared to VMs, being part of the research in [21]–[24].

Further, the authors in [7] and [9] focus on live migration of industrial control services. Even if both investigations use PPM for live migration, [7] transfers the complete memory from the source to the target system, while [9] applies a more user-centric approach where only state-variables are transferred that were defined by the user beforehand. This makes the approach super fast, efficient, and platform independent, but requires knowledge of the control process. On the other hand, the approach of [7] does not require insights of the control process itself, but is a major achievement compared to inter-copy, pre-copy, post-copy, and hybrid C/R. The only drawback, however, is that the process is still checkpointed. This means that it is intentionally stopped before the last memory is transmitted to the target system and resumed.

VI. NOVEL CONCEPT

In order to further minimize the downtime a novel concept is proposed in this section.

Here, two different hosts are assumed, where one of the hosts is the source (SRC) and the other the destination (DST) of the live migration, as shown in Fig. 2. Additionally, both hosts provide the compute resources r_{SRC} and r_{DST} . Moreover, a

migration controller, a network controller and a set of so-called control containers run on both of the devices. For the control containers, a resource consumption of $c_1, c_2, \dots, c_n$ is assumed.

Further, at least one Cyber-Physical System (CPS) that can be any industrial device, sends data to a control container that is located on *SRC*. The control process itself can be described by a state vector $\underline{x}(t)$ with n states, denoted by $x_1(t), \dots, x_n(t)$ and an output vector $\underline{y}(t)$ with k outputs denoted by $y_1(t), \dots, y_k(t)$, respectively.

The convergence criteria can be described by deviation δ_y of the output vectors of *SRC*, denoted by $y_{a,1}, \dots, y_{a,k}$ and *DST*, denoted by $y_{b,1}, \dots, y_{b,k}$. Only if all $\delta_{y,i} = 0$ or near sufficient $\delta_{y,i} \leq \eta_{y,i}$ a handover is initiated:

$$\delta_y(t) = \underline{y}_a(t) - \underline{y}_b(t) = \begin{pmatrix} y_{a,1}(t) - y_{b,1}(t) \\ y_{a,2}(t) - y_{b,2}(t) \\ \vdots \\ y_{a,k}(t) - y_{b,k}(t) \end{pmatrix} = \begin{pmatrix} \delta_{y,1}(t) \\ \delta_{y,2}(t) \\ \vdots \\ \delta_{y,k}(t) \end{pmatrix} \quad (1)$$

The live migration process is described in Algorithms 1 & 2.

Algorithm 1 Migration process *Option 1*

Require: Migration request, Migration target *DST*

Ensure: $\sum_{k=0}^n c_k + c_{n+1} < r_{DST}$

- 1: Connection setup b/w *SRC* and *DST*
 - 2: Start packet buffer on *DST*
 - 3: Forward input data packets to *DST*
 - 4: Start state zero container on *DST*
 - 5: Forward output data $\underline{y}$ to *DST*
 - 6: **while** $\delta_{y,i} \not\leq \eta_{y,i}, \forall i$ **do**
 - 7: Send $\underline{x}$ to *DST*
 - 8: **while** packet buffer > 0 **do**
 - 9: Read next packet
 - 10: **end while**
 - 11: **end while**
 - 12: Initiate handover ▷ Success
-

If a live migration from *SRC* to *DST* is triggered, it first has to be checked if an additional control container with the compute resources c_{n+1} can be deployed on the target, or if the targets resources r_{DST} are exceeded. If this is ensured, the connection between *SRC* and *DST* is established and a packet buffer is started on *DST* by the migration controller. Further, all incoming packets that are addressed to the specific container on *SRC* are copied, timestamped, and redirected to *DST*. These packets are cached in the packet buffer.

Next, a control container is started on *SRC* by its migration controller. Dependant on the requirements of the maximum allowed migration time, two options are possible:

- 1) Start of a state zero container and transfer of the state map $\underline{x}(t)$
- 2) Start of a backup container at the beginning of the control process and C/R to *DST*

While *Option 1* has a lower migration time, *Option 2* can be applied non-invasive, which means that no process knowledge

Algorithm 2 Migration process *Option 2*

Require: Migration request, Migration target *DST*

Ensure: $\sum_{k=0}^n c_k + c_{n+1} < r_{DST}$

- 1: Connection setup b/w *SRC* and *DST*
 - 2: Start packet buffer on *DST*
 - 3: Forward input data packets to *DST*
 - 4: Stop redundant control container on *SRC*
 - 5: Checkpoint stopped control container on *SRC*
 - 6: Transmit and restart control container on *DST*
 - 7: Forward output data $\underline{y}$ to *DST*
 - 8: **while** packet buffer > 0 **do**
 - 9: Read next packet
 - 10: **end while**
 - 11: **if** $\delta_{y,i} \leq \eta_{y,i}, \forall i$ **then**
 - 12: Initiate handover ▷ Success
 - 13: **else if** $\delta_{y,i} \not\leq \eta_{y,i}, \forall i$ **then**
 - 14: Restart migration process ▷ Error
 - 15: **end if**
-

is required using this option. This is possible, since an identical container is started at the same starting point as the first container and receives all packets that the main control container receives from the beginning. Thus, any C/R approach can be used to migrate this container from *SRC* to *DST*.

After the two control container instances are running on *SRC* to *DST* the output data packets that contain $\underline{y}$ of *SRC* are forwarded to *DST*.

Then, for *Option 1*, the state map $\underline{x}$ of *SRC* is forwarded to *DST*. Subsequently both the state map and all data packets from the buffer with a timestamp equal to or greater than that of the state map are loaded and replayed in sequence. Afterwards it is checked if the pre-defined convergence criteria of Eq. 2 are fulfilled.

On the other hand, for *Option 2*, no state map $\underline{x}$ is required. Hence, all data packets from the buffer with a timestamp equal to or greater than that of the checkpoint are loaded and replayed in sequence and the convergence criteria are checked. If $\delta_{y,i} \leq \eta_{y,i}, \forall i$ the handover is initiated by the migration controller.

Even if the second option increases the migration time, it can be advantageous to have at least one backup container running in case of any error that can occur. In a realistic scenario, it makes also sense to deploy this backup container on another physical system than the main container. This makes the system more robust against hardware errors and faults in the communication link. If the backup has been proactively deployed to a potential target system for live migration and that system is selected as destination, the process itself does not need to be migrated.

VII. TESTBED AND EVALUATION

In this section, our concept is evaluated. Therefore, a testbed consisting of real hardware (Sec. VII-A) as well as an exemplary process (Sec. VII-B) are proposed and the planned measurements are introduced (Sec. VII-C). Afterwards, the measurements are evaluated (Sec. VII-D).

Tab. III
HARDWARE CONFIGURATIONS

Equipment	QTY	Specification
Mini PC	3	Intel Core i7-8809G, 2x16 GB DDR4, Intel i210-AT & i219-LM Gigabit NICs, Debian 10.9.0 64 bit, Linux 4.19.0-16-rt
Network Switch	1	8-Port Gigabit Ethernet Switch
TSN Evaluation Kit	1	RAPID-TSNEK-V0001, IEEE 802.1AS-REV

A. Testbed

In order to implement our concept in a realistic setup several hardware components are required, which are listed in Tab. III.

First, two processing nodes are required that serve as source and target system for the migration process. Here, small sized mini PCs were used, because they could easily be integrated in existing industrial plants and serve as potential edge servers. Further, mini PCs are very often used for similar investigations. Thus a good comparability between the results is given. Additionally, both mini PCs are running on similar Linux OS and RT Kernel. The latter is extremely important to achieve the required determinism of RT applications. In order to perform the planned measurements of T_M and T_D for both options of our concept, *Docker* and *Checkpoint/Restore In Userspace (CRIU)* have been installed on the hosts. The latter is a software tool, widely used for inter-copy C/R of both container and VMs.

Moreover, both hosts have two Network Interface Cards (NICs). This allows for the physical separation of RT network that is required for the manufacturing process itself and a non RT network for the integration in the factory network that provides management and orchestration interfaces and possibly also internet connection. In this work, the second network interface was used for the connection to a time master that synchronizes the clocks of both hosts, using *linuxptp* open source software module. Here it was already shown that the synchronization error is $\ll 1\mu s$ [9]. Thus, this error can be neglected. Furthermore, both hosts are connected to each other and the CPS via a commercial off-the-shelf (COTS) network switch. A similar mini PC is used as CPS.

B. Example Process

That the proposed concept can be tested properly, an exemplary process is specified (see Fig. 3). Thus, a Markov chain is used that consists of the states s_m , with $m = 0, \dots, 6$ and the transitions t_{ij} , where i is the current state, and j is the next state. Further, the state is updated once every 5 milliseconds and the probabilities for the transitions that are possible in the corresponding state are equally distributed. The transition probabilities are depicted in Eq. 2.

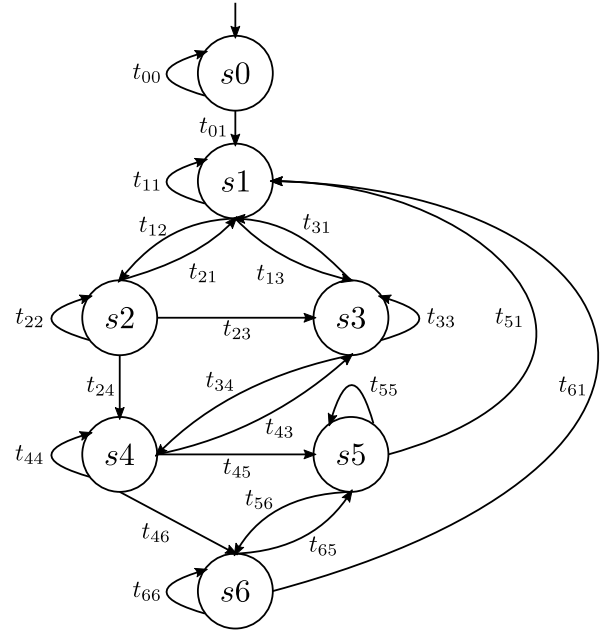


Fig. 3. Exemplary process including states and transitions.

$$P = \begin{pmatrix} 0.5 & 0.5 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0.3 & 0.3 & 0.3 & 0 & 0 & 0 \\ 0 & 0.25 & 0.25 & 0.25 & 0.25 & 0 & 0 \\ 0 & 0.3 & 0 & 0.3 & 0.3 & 0 & 0 \\ 0 & 0 & 0 & 0.25 & 0.25 & 0.25 & 0.25 \\ 0 & 0.3 & 0 & 0 & 0 & 0.3 & 0.3 \\ 0 & 0.3 & 0 & 0 & 0 & 0.3 & 0.3 \end{pmatrix} \quad (2)$$

C. Measurements

In order to implement the novel concept in real industrial applications, the performance is decisive. Especially, if the process downtime T_D is too high, a live migration under normal operation of the plant is not possible. To be clear, the process downtime has to be at least smaller as the update interval T_U , in which the CPS requires novel control information. Thus, the following condition has to hold:

$$T_D \leq T_U \quad (3)$$

Moreover, the duration of the whole migration process is relevant, since it specifies the dynamic of applications that require process migrations. Thus, if the migration time T_M is very high, this could be a show stopper, even if Eq. 3 is fulfilled. Hence, Eq. 4 and Eq. 5 express the migration times for *Option 1* and *Option 2*, respectively:

$$T_{M|Option\ 1} = T_{image|prep} + T_{create} + T_{start} + T_D \quad (4)$$

$$T_{M|Option\ 2} = T_{M|Option\ 1} + T_{c/r} \quad (5)$$

Looking into Eq. 4, it can be seen that the migration time

is the sum of image preparation $T_{image|prep}$, creation and start of the container including the process to be migrated (T_{create}, T_{start}), as well as the process downtime (T_D). In order to describe the migration time for *Option 2*, the time that is consumed by CRIU for applying C/R mechanism of the backup container ($T_{c/r}$) has to be added (see Eq. 5). Furthermore, for the image preparation process there are two basic scenarios considered:

- 1) The image is already located on *DST*.
- 2) The image has to be transferred to *DST*.

If the image is already located on *DST*, it can be assumed that $T_{image|prep} = 0$ since no data has to be transferred over the network. However, maintaining the images of all services on each possible node would result in a high space consumption on the disks of the nodes as well as to inconsistency if any change to an image is done. Thus, this scenario is only realistic, if the corresponding process was already located on this hardware node. This could be the case, if an AGV, such as a drone, is operating at the boundary of two factory floors, and the process needs to be live migrated at high frequency. On the other hand, in the second scenario, the image has to be transferred to *DST*. Here, it is assumed that the image is already located at least inside the factory network. This is realistic since the container including the process is already running on *SRC* and the corresponding image had to be build in advance. Here, two more possibilities are included:

- 1) The image is located at any data server in the central cloud of the factory.
- 2) The image is located at *SRC*.

Since the provision of all images of the processes that are running on one system might not be efficient in terms of space consumption of the disk, it could be beneficial to build and maintain the image only on a centralized data server that is part of the factory cloud. Therefore, the time T_{pull} for the pull of the image from the factory cloud to the *DST* is assumed.

If the image is available at *SRC*, it is beneficial to send the image directly to *DST*. Here, two widely used protocols for file transfer, namely Secure Copy Protocol (SCP) and Remote Copy Protocol (RCP), are integrated. These protocols offer simple security mechanisms by the requirement of user credentials and encryption. Since this results in an overhead that could significantly increase the migration time, a customized server-client communication for exchanging the image using Transmission Control Protocol (TCP) was also realized.

D. Evaluation

First, the results on the identified process downtime, which are depicted in Fig. 4, are analysed. It can be seen that T_D lies consequently under 1ms. Looking deeper into the data set, it can be concluded that both median and average value of T_D lie in the area of 0.2ms. This means that also use cases can supported that have very stringent requirements on latency and determinism. To be clear, use cases that require update times of 0.2ms and more can be live migrated without any visible effects.

Next, Fig. 5 displays the second metric T_M . The readings

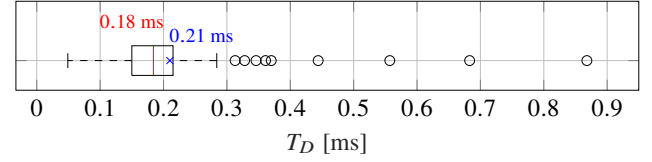


Fig. 4. Identified process downtime T_D during a live migration process using our concept.

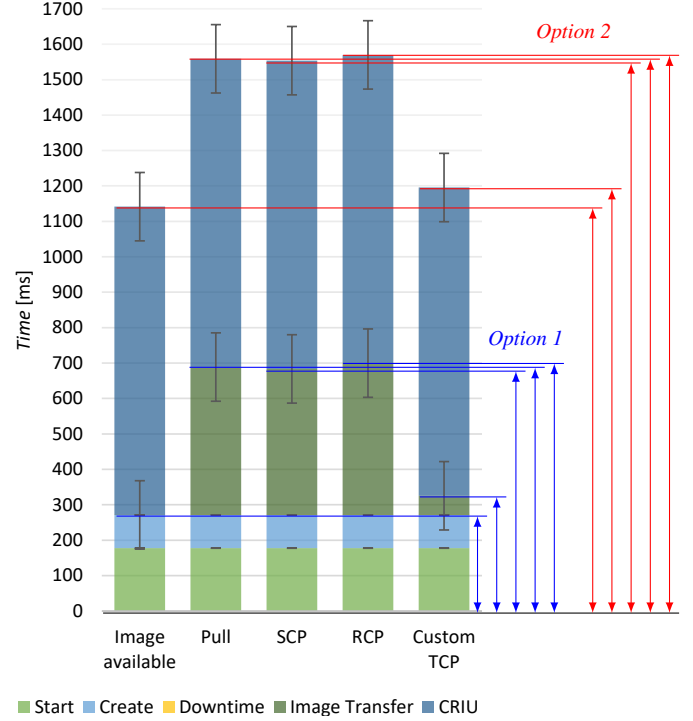


Fig. 5. Migration times T_M for *Option 1* and *Option 2* based on different image transfer methods.

of T_M indicate that it lies consequently below 1s for *Option 1* and below 2s for *Option 2*. This means that a live migration can be performed more than once per second for *Option 1* and more than once every 2 seconds for *Option 2*. Moreover, the biggest fraction for the migration time of *Option 2* is the time consumed for applying C/R method. This confirms our effort in developing a novel concept for live migration, since most existing approaches are still using this approach for live migration. Thus, T_{CRIU} would be a fraction of the process downtime that is 1000 times higher compared to our approach. Even if T_{CRIU} is only a fraction of T_M in our approach, the migration time without using *Option 2* is significantly lower. Thus, if possible, *Option 1* should be applied, if a high migration dynamic is required. Further, the overhead of SCP and RCP can be observed that is comparable to a pull of the image located at a central data server. Reasons for this overhead were already mentioned before. Hence, it can be concluded that a customized TCP server-client communication is the preferred

method. Using this method, a migration time of 325ms can be reached that is only about 15% higher compared to the scenario, where the image is directly available on the target system. Here, a migration time of ≈ 270 ms can be realized. Thus, live migrations of processes can be performed up to 3-4 times per second, using our optimized approach.

VIII. CONCLUSION

In this paper, a novel concept for a downtime optimized live migration that is targeted for industrial RT services was investigated. Therefore, the state-of-the-art was introduced and the different approaches based on specified metrics that are most relevant were analysed. Additionally, related work was proposed and existing challenges were discussed. In order to improve these challenges a novel concept was proposed and evaluated based on a testbed and exemplary process. The results clearly show that live migration using this concept only causes a process downtime in sub-milliseconds range. Hence, the proposed concept is able to fulfill mission-critical use cases that have very high demands on determinism and latency.

Furthermore, the migration time lies consequently below 1 second for the live migration of time invariant state-zero process controllers and below 2 seconds for using C/R on a backup container. This means that a live migration of a mission-critical service can be performed at least once per second, or every 2 seconds, respectively. Using a simplified transfer of the image, or is the image already available on the target node, the migration time can be further reduced below 300ms. Thus, a live migration can be performed multiple times per second. This allows for highly flexible control services that are prerequisite for realizing Industry 4.0 scenarios.

ACKNOWLEDGMENT

This research was supported by the German Federal Ministry for Economic Affairs and Climate Action (BMWK) within the project FabOS under grant number 01MK20010C. The responsibility for this publication lies with the authors.

REFERENCES

- [1] E. Hozdić, "Smart factory for industry 4.0: A review," *International Journal of Modern Manufacturing Technologies*, vol. 7, no. 1, pp. 28–35, 2015.
- [2] M. Gundall, J. Schneider, H. D. Schotten, M. Aleksy, *et al.*, "5G as Enabler for Industrie 4.0 Use Cases: Challenges and Concepts," in *2018 IEEE 23rd International Conference on Emerging Technologies and Factory Automation (ETFA)*, vol. 1, Sep. 2018, pp. 1401–1408. DOI: 10.1109/ETFA.2018.8502649.
- [3] H. Lasi, P. Fettke, H. Kemper, T. Feld, *et al.*, "Industry 4.0," *Business & information systems engineering*, pp. 239–242, 2014.
- [4] M. Gundall, M. Strufe, H. D. Schotten, P. Rost, *et al.*, "Introduction of a 5G-Enabled Architecture for the Realization of Industry 4.0 Use Cases," *IEEE Access*, vol. 9, pp. 25 508–25 521, 2021. DOI: 10.1109/ACCESS.2021.3057675.
- [5] W. Jiang, B. Han, M. A. Habibi, and H. D. Schotten, "The road towards 6G: A comprehensive survey," *IEEE Open Journal of the Communications Society*, vol. 2, pp. 334–366, 2021.
- [6] A. Strunk, "Costs of virtual machine live migration: A survey," in *2012 IEEE Eighth World Congress on Services*, IEEE, 2012, pp. 323–329.
- [7] K. Govindaraj and A. Artemenko, "Container live migration for latency critical industrial applications on edge computing," in *2018 IEEE 23rd International Conference on Emerging Technologies and Factory Automation (ETFA)*, IEEE, vol. 1, 2018, pp. 83–90.
- [8] T. Goldschmidt, S. Hauck-Stattelmann, S. Malakuti, and S. Grüner, "Container-based architecture for flexible industrial control applications," *Journal of Systems Architecture*, vol. 84, pp. 28–36, 2018.
- [9] M. Gundall, C. Glas, and H. D. Schotten, "Feasibility Study on Virtual Process Controllers as Basis for Future Industrial Automation Systems," in *2021 22nd IEEE International Conference on Industrial Technology (ICIT)*, 2021.
- [10] E. Al-Shaer, Q. Duan, and J. H. Jafarian, "Random Host Mutation for Moving Target Defense," in *Security and Privacy in Communication Networks*, A. D. Keromytis and R. Di Pietro, Eds., Berlin, Heidelberg: Springer Berlin Heidelberg, 2013, pp. 310–327.
- [11] M. Gundall, J. Stegmann, C. Huber, and H. D. Schotten, "Towards Organic 6G Networks: Virtualization and Live Migration of Core Network Functions," in *Mobile Communication - Technologies and Applications; 25. ITG-Symposium*, Oct. 2021, pp. 1–5.
- [12] S. Grüner, S. Malakuti, J. Schmitt, T. Terzimehic, *et al.*, "Alternatives for Flexible Deployment Architectures in Industrial Automation Systems," in *2018 IEEE 23rd International Conference on Emerging Technologies and Factory Automation (ETFA)*, vol. 1, Sep. 2018, pp. 35–42. DOI: 10.1109/ETFA.2018.8502526.
- [13] Adrian Reber, *Process Migration in a Parallel Environment*. Stuttgart, 2016. DOI: 10.18419/opus-8791.
- [14] C. Puliafito, C. Vallati, E. Mingozzi, G. Merlino, *et al.*, "Container Migration in the Fog: A Performance Evaluation," *Sensors*, vol. 19, no. 7, 2019. DOI: 10.3390/s19071488.
- [15] H. Nie, P. Li, H. Xu, L. Dong, *et al.*, "Research on Optimized Pre-copy Algorithm of Live Container Migration in Cloud Environment," in *Parallel Architecture, Algorithm and Programming*, Chen, Guoliang and Shen, Hong and Chen, Mingrui, Ed., Singapore: Springer Singapore, 2017, pp. 554–565.
- [16] M. R. Hines, U. Deshpande, and K. Gopalan, "Post-copy live migration of virtual machines," *ACM SIGOPS operating systems review*, vol. 43, no. 3, pp. 14–26, 2009.
- [17] L. Ma, S. Yi, and Q. Li, "Efficient Service Handoff across Edge Servers via Docker Container Migration," in *Proceedings of the Second ACM/IEEE Symposium on Edge Computing*, ser. SEC '17, New York, NY, USA: Association for Computing Machinery, 2017. DOI: 10.1145/3132211.3134460.
- [18] S. Wang, J. Xu, N. Zhang, and Y. Liu, "ASurvey on Service Migration in Mobile Edge Computing," *IEEE Access*, vol. 6, pp. 23 511–23 528, 2018. DOI: 10.1109/ACCESS.2018.2828102.
- [19] C. Clark, K. Fraser, S. Hand, J. G. Hansen, *et al.*, "Live migration of virtual machines," in *Proceedings of the 2nd conference on Symposium on Networked Systems Design & Implementation-Volume 2*, 2005, pp. 273–286.
- [20] R. Torre, R.-S. Schmoll, F. Kemser, H. Salah, *et al.*, "Benchmarking Live Migration Performance Under Stressed Conditions," in *2021 IEEE 18th Annual Consumer Communications & Networking Conference (CCNC)*, IEEE, 2021, pp. 1–2.
- [21] W. Felter, A. Ferreira, R. Rajamony, and J. Rubio, "An updated performance comparison of virtual machines and Linux containers," in *2015 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, Mar. 2015, pp. 171–172. DOI: 10.1109/ISPASS.2015.7095802.
- [22] M. Gundall, D. Reti, and H. D. Schotten, "Application of Virtualization Technologies in Novel Industrial Automation: Catalyst or Show-Stopper?" In *2020 IEEE 18th International Conference on Industrial Informatics (INDIN)*, IEEE, vol. 1, 2020, pp. 283–290.
- [23] A. Moga, T. Sivanthi, and C. Franke, "OS-Level Virtualization for Industrial Automation Systems: Are We There Yet?" In *Proceedings of the 31st Annual ACM Symposium on Applied Computing*, ser. SAC '16, Pisa, Italy: Association for Computing Machinery, 2016, pp. 1838–1843. DOI: 10.1145/2851613.2851737. [Online]. Available: <https://doi.org/10.1145/2851613.2851737>.
- [24] A. M. Joy, "Performance comparison between Linux containers and virtual machines," in *2015 International Conference on Advances in Computer Engineering and Applications*, 2015, pp. 342–346.